

> LAB 1

↳ 6 cells hidden

> LAB 2

↳ 11 cells hidden

> LAB 3

▶ ↳ 4 cells hidden

v LAB 4

```
import pandas as pd

file_path = '/content/spam_or_not_spam.csv'
df = pd.read_csv(file_path)
display(df.head())
```

	email	label	
0	date wed NUMBER aug NUMBER NUMBER NUMBER NUMB...	0	
1	martin a posted tassos papadopoulos the greek ...	0	
2	man threatens explosion in moscow thursday aug...	0	
3	klez the virus that won t die already the most...	0	
4	in adding cream to spaghetti carbonara which ...	0	

```
from sklearn.feature_extraction.text import TfidfVectorizer

text_column_name = 'email'

tfidf_vectorizer = TfidfVectorizer(max_features=5000) # You can adjust max_features as needed

X_features = tfidf_vectorizer.fit_transform(df[text_column_name].astype(str))

print(f"Shape of the extracted features: {X_features.shape}")
```

Shape of the extracted features: (3000, 5000)

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix

import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = df.dropna(subset=["email", "label"])

X = df["email"]
y = df["label"]

print("Dataset shape:", df.shape)
```

Dataset shape: (2999, 2)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words='english',
    ngram_range=(1,2)
)

X_train = vectorizer.fit_transform(X_train).toarray()
X_test = vectorizer.transform(X_test).toarray()

print("X_train shape:", X_train.shape)
```

```
X_train shape: (2399, 5000)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("X_train type after re-split:", type(X_train))
print("X_test type after re-split:", type(X_test))
```

```
X_train type after re-split: <class 'pandas.core.series.Series'>
X_test type after re-split: <class 'pandas.core.series.Series'>
```

```
vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words='english',
    ngram_range=(1,2)
)

X_train = vectorizer.fit_transform(X_train).toarray()
X_test = vectorizer.transform(X_test).toarray()

print("X_train shape after vectorization:", X_train.shape)
print("X_test shape after vectorization:", X_test.shape)
```

```
X_train shape after vectorization: (2399, 5000)
X_test shape after vectorization: (600, 5000)
```

```
input_size = X_train.shape[1]

np.random.seed(42)

W = np.random.randn(input_size, 1) * 0.01
b = np.zeros((1,1))

learning_rate = 0.1
epochs = 500
```

```
def sigmoid(Z):
    return 1 / (1 + np.exp(-Z))
```

```
def forward(X):
    Z = np.dot(X, W) + b
    A = sigmoid(Z)
    return A
```

```
def compute_loss(A, y):
    m = y.shape[0]
```

```
epsilon = 1e-8

loss = - (1/m) * np.sum(
    y*np.log(A+epsilon) +
    (1-y)*np.log(1-A+epsilon)
)
return loss
```

```
def backward(X, y, A):
    m = X.shape[0]

    dZ = A - y
    dW = (1/m) * np.dot(X.T, dZ)
    db = (1/m) * np.sum(dZ)

    return dW, db
```

```
loss_history = []
accuracy_history = []

# Convert y_train (pandas Series) to a NumPy array and reshape it to (m, 1)
# This ensures consistent shapes with the model's output A for element-wise operations.
y_train_np = y_train.values.reshape(-1, 1)

for i in range(epochs):

    # Forward
    A = forward(X_train)

    # Loss
    # Pass the NumPy array y_train_np with shape (m, 1)
    loss = compute_loss(A, y_train_np)
    loss_history.append(loss)

    # Accuracy on training set
    predictions = (A >= 0.5).astype(int)
    # Compare predictions (shape (m, 1)) with y_train_np (shape (m, 1))
    train_acc = np.mean(predictions == y_train_np)
    accuracy_history.append(train_acc)

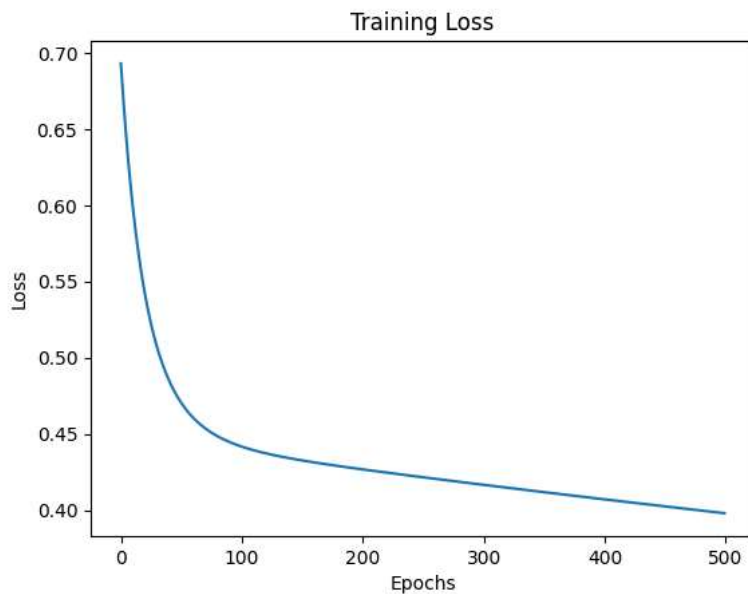
    # Backward
    # Pass the NumPy array y_train_np with shape (m, 1)
    dW, db = backward(X_train, y_train_np, A)

    # Update
    W -= learning_rate * dW
    b -= learning_rate * db

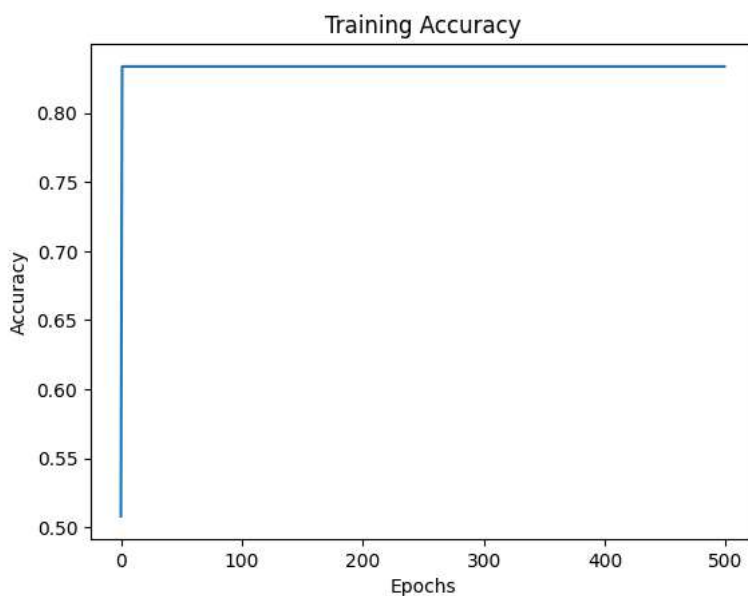
    if i % 100 == 0:
        print(f"Epoch {i}, Loss: {loss}, Accuracy: {train_acc}")
```

```
Epoch 0, Loss: 0.6930196735975684, Accuracy: 0.5081283868278449
Epoch 100, Loss: 0.44158810880860666, Accuracy: 0.8336807002917882
Epoch 200, Loss: 0.4266464542217769, Accuracy: 0.8336807002917882
Epoch 300, Loss: 0.4165259168842215, Accuracy: 0.8336807002917882
Epoch 400, Loss: 0.4069859087523394, Accuracy: 0.8336807002917882
```

```
plt.figure()
plt.plot(loss_history)
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()
```



```
plt.figure()
plt.plot(accuracy_history)
plt.title("Training Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.show()
```



```
def predict(X, threshold=0.2):
    A = forward(X)
    return (A >= threshold).astype(int)
```

```
y_pred = predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
```

```
print("F1 Score:", f1)
```

```
Accuracy: 0.9233333333333333  
Precision: 1.0  
Recall: 0.54  
F1 Score: 0.7012987012987013
```

```
print("\nClassification Report:\n")  
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0	0.92	1.00	0.96	500
1	1.00	0.54	0.70	100
accuracy			0.92	600
macro avg	0.96	0.77	0.83	600
weighted avg	0.93	0.92	0.91	600